

Data Management Technologies

Unit 5 - Consolidation Task – Individual Exercises

1. Understanding Relations

Given the sets $S = \{A, B\}$ and $T = \{1, 2, 3\}$, write out the Cartesian product.

	1	2	3
A	(A, 1)	(A, 2)	(A, 3)
B	(B, 1)	(B, 2)	(B, 3)

Identify subsets that qualify as valid relations.

There are 64 possible subsets (calculated as 2^6).

Examples are:

- $\{(A,1)\}$
- $\{(A,1), (B,2)\}$
- $\{(A,1), (A,2), (A,3), (B,1), (B,2), (B,3)\}$

2. Schema Construction

Design schemas for:

Students(name: string, sid: string, age: integer)

Courses(cid: string, title: string, credits: integer)

Insert three instances (rows) for each schema.

Schema Definition

Students Schema:

```
CREATE TABLE Students (  
  Name VARCHAR (100);  
  sid VARCHAR (6) PRIMARY KEY;  
  Age INTEGER  
);
```

Instances:

INSERT INTO Students **VALUES** ('John Smith', 'SD001', 18);
INSERT INTO Students **VALUES** ('Alice Brown', 'SD002', 21);
INSERT INTO Students **VALUES** ('James White', 'SD003', 24);

Name	sid	Age
John Smith	SD001	20
Alice Brown	SD002	22
James White	SD003	19

Courses Schema:

CREATE TABLE Courses (
cid **VARCHAR** (4) **PRIMARY KEY**;
Title **VARCHAR** (100);
Credits **INTEGER** (3)
);

Instances:

INSERT INTO Courses **VALUES** ('CS01', 'Mathematics', 60);
INSERT INTO Courses **VALUES** ('CS02', 'Art History', 30);
INSERT INTO Courses **VALUES** ('CS03', 'Data Science', 30);

Title	cid	Credits
Mathematics	CS01	60
Art History	CS02	30
Data Science	CS03	30

3. Selection and Projection Operators

Given relation R(A, B, C, D) , write SQL queries to:

- A) Select rows where A > 10 .
SELECT * FROM table **WHERE** A > 10;
- B) Select only columns B and C .
SELECT B, C **FROM** table
- C) Combine selection and projection to filter A > 10 and display B , C
SELECT B, C **FROM** table **WHERE** A > 10;

4. Set Operations

Create two tables, R and S with overlapping rows.

```
CREATE TABLE R (  
    ID INTEGER (4) PRIMARY KEY;  
    Value VARCHAR (10);  
);
```

```
CREATE TABLE S (  
    ID INTEGER (4) PRIMARY KEY;  
    Value VARCHAR (10);  
);
```

```
INSERT INTO R VALUES (001,'John'), (002, 'James'), (003, 'Mary');  
INSERT INTO S VALUES (002, 'James'), (004, 'Phillip'), (005, 'Stephen');
```

Write SQL queries for:

Union of R and S

```
SELECT * FROM R  
UNION  
SELECT * FROM S
```

Intersection of R and S

```
SELECT * FROM R  
INTERSECT  
SELECT * FROM S
```

Difference of R and S

```
SELECT * FROM R  
EXCEPT  
SELECT * FROM S
```

5. Cartesian Product and Joins

a.) Write a Cartesian product query for R (A , B) and S(C, D) .

```
SELECT * FROM R CROSS JOIN S
```

b.) Perform an inner join on tables Students (name, sid, cid) and Colleges (cid, cname) using the column cid.

(Inner join returns rows where the join condition matches in both tables)

```
SELECT Students.name, Students.sid, Colleges.cname
FROM Students
INNER JOIN Colleges ON Students.cid = Colleges.cid
```

c.) Extend the query to a left outer join.

Left outer join returns all students, including those without a matching college. In these cases, the college column returns NULL.

```
SELECT Students.name, Students.sid, Colleges.cname
FROM Students
LEFT OUTER JOIN Colleges ON Students.cid = Colleges.cid;
```

6. Table Creation and Manipulation

Create the tables Accounts and Customers from the given schema using SQL.
Insert, update, and delete data as described in Task 1 of the Workshop slides.

1. Are there suitable candidate keys or primary keys in these relations?

Candidate keys: acctNumber

(This is the only suitable candidate/primary key as it is the only unique key).

2. What are the attributes and the tuples of each relation?

Accounts:

Attributes (columns) - acctNumber, type, balance

Tuples (rows)-each of the three records

Customers:

Attributes – firstName, lastName, cID, account.

Tuples (rows) - each of the three records.

3. Are these related in some way? If not, how would you relate them?

The attributes acctNumber and account correspond.

4. Indicate the component(s) of one record from each relation.

Accounts: ('AC12345', 'savings', 1200)

Customers: ('Robert', 'Ayala', '901-222', 'AC12345')

5. Create both tables using SQL commands and appropriate datatypes.

(Done in SQL script, included in appendix below)

6. Insert two or three records to each table in any order

(Done in SQL script included in appendix below)

7. Delete one record in each table.

(Done in SQL script included in appendix below)

8. What happens if you add column Age to table Accounts?

A new column is created, which includes NULL in each field, as no values have been entered yet.

9. If you have added an attribute Age to table Accounts, how would you populate any missing values?

With the UPDATE keyword. For example:

```
UPDATE Accounts SET age = 35 WHERE acctNumber = 'AC34567';
```

10. Create a query from table Accounts that only has records with more than 500 pounds deposited.

```
SELECT * FROM Accounts WHERE balance > 500;
```

11. Create a query that only returns the last name of table Customers and their account number.

```
SELECT lastName, account FROM Customers;
```

12. How would you create a table that has the customers' data and bank balance, as in the following example:

This can be done by using an INNER JOIN to merge the tables (as the tuples in each table have a corresponding entry in the other table)

```
CREATE TABLE CustomerAccounts AS  
SELECT * FROM Customers  
JOIN Accounts ON Customers.account = Accounts.acctNumber;
```

7. Joining and Merging

a.) Write a query that combines Customers and Accounts to display:
Customer's last name.
Their account number and balance.

```
SELECT * FROM Customers  
JOIN Accounts ON Customers.account = Accounts.acctNumber;
```

b.) Merge the Loan and Borrower tables using inner, left, and full outer joins.

-- INNER JOIN: only matching rows

```
SELECT * FROM Borrower`  
  
INNER JOIN Loan ON Borrower.loanNumber = Loan.loanNumber;
```

-- LEFT JOIN: all Table2 rows, matched with Table3 rows (**NULL** if no match)

```
SELECT * FROM Borrower  
  
LEFT JOIN Loan ON Borrower.loanNumber = Loan.loanNumber;
```

-- FULL OUTER JOIN: all rows from both tables (**NULL** where no match)

```
SELECT * FROM Borrower  
  
FULL OUTER JOIN Loan ON Borrower.loanNumber = Loan.loanNumber;
```

Appendix: SQL Script for Question 6

DROP TABLE Accounts;

DROP TABLE Customers;

CREATE TABLE Accounts

(acctNumber VARCHAR (25),

type VARCHAR (255) NOT NULL,

balance INT NOT NULL

);

CREATE TABLE Customers

(firstName VARCHAR (25),

lastName VARCHAR (25),

cID VARCHAR NOT NULL,

account VARCHAR NOT NULL

);

INSERT INTO Accounts (acctNumber, type, balance) VALUES ('AC12345', 'Savings', 12000);

INSERT INTO Accounts (acctNumber, type, balance) VALUES ('AC23456', 'Checking', 1000);

INSERT INTO Accounts (acctNumber, type, balance) VALUES ('AC34567', 'Savings', 25);

INSERT INTO Customers (firstName, lastName, cID, account) VALUES ('Robert', 'Ayala', '901-222', 'AC12345');

```
INSERT INTO Customers (firstName, lastName, cID, account) VALUES ('Anna', 'Marini', '805-333', 'AC12345');
```

```
INSERT INTO Customers (firstName, lastName, cID, account) VALUES ('Anna', 'Marini', '805-333', 'AC34567');
```

-- Q6) Insert two or three records to each table in any order

```
INSERT INTO Accounts (acctNumber, type, balance) VALUES ('AC45678', 'Savings', 100);
```

```
INSERT INTO Accounts (acctNumber, type, balance) VALUES ('AC56789', 'Checking', 5000);
```

```
INSERT INTO Accounts (acctNumber, type, balance) VALUES ('AC67891', 'Savings', 200);
```

```
INSERT INTO Customers (firstName, lastName, cID, account) VALUES ('James', 'Brown', '934-444', 'AC45678');
```

```
INSERT INTO Customers (firstName, lastName, cID, account) VALUES ('Norton', 'Grey', '768-111', 'AC56789');
```

```
INSERT INTO Customers (firstName, lastName, cID, account) VALUES ('Lilian', 'Matthews', '354-222', 'AC67891');
```

-- Q7) Delete one record on each table.

```
DELETE FROM Accounts WHERE acctnumber = 'AC45678';
```

```
DELETE FROM Customers WHERE cID = '768-111';
```

--Q8) Adding column age (as an integer data type) to accounts

```
ALTER TABLE Accounts ADD age int;
```

--Q9) Populating age column

```
UPDATE Accounts SET age = 35 WHERE acctNumber = 'AC34567';
```

-- Q10) Query accounts

```
SELECT * FROM Accounts WHERE balance > 500;
```


--Q11) Query customers

```
SELECT lastName, account FROM Customers;
```

-- Displaying the tables.

```
SELECT * FROM Accounts;
```

```
SELECT * FROM Customers
```